

EECS 182 Homework 7 Code Report

user

Your task is to produce the solution to my machine learning homework showing intermediate reasoning. Avoid hallucinating or making up definitions if information is missing, explicitly say so. Answer with clear math notation and concise justification. Do not skip derivation steps. If the problem contains multiple subparts (a), (b), (c), answer each clearly. I am going to give you functions with TODO. Fill the function. Don't overengineer solutions. Follow tips

ChatGPT

Perfect—ready when you are.

Please paste (or upload) your assignment and the **exact code file(s)** with the ``# TODO`` blocks. Include any “Tips” section from the handout so I can follow it.

To move fast, drop these in your next message:

- 1) **Problem text** (all subparts (a),(b),(c)...).
- 2) **Code with TODOs** inside triple backticks, e.g.

```
```python
def foo(x):
 # TODO: implement ...
 pass
```
```

- 3) Any constraints (allowed libs, expected shapes, runtime limits, grading/autograder notes).

What you'll get back in one reply:

- A **step-by-step solution** with clear math and concise justifications (no skipped derivations; I'll say explicitly if something is missing).
- The **filled-in functions only**, keeping changes minimal (no over-engineering).
- Brief inline comments aligning code with the math.

Send the files/blocks and I'll dive right in.

user

1.A: implementing a RNN layer

Consider using Pytorch's

[nn.Linear](<https://pytorch.org/docs/stable/generated/torch.nn.Linear.html#torch.nn.Linear>). You can implement this with either one Linear layer or two. If you use two, remember that you only need to include a bias term for one of the linear layers.

```
class RNNLayer(nn.Module):
    def __init__(self, input_size, hidden_size, nonlinearity=th.tanh):
        """
        Initialize a single RNN layer.

        Inputs:
        - input_size: Data input feature dimension
        - hidden_size: RNN hidden state size (also the output feature dimension)
        - nonlinearity: Nonlinearity applied to the rnn output
        """
        super().__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.nonlinearity = nonlinearity

#####
#####
# TODO: Initialize any parameters your class needs. #
#####
#####
#####
#####
#                               END OF YOUR CODE                               #
```

```
#####  
#####
```

```
def forward(self, x):
```

```
    """
```

```
    RNN forward pass
```

```
    Inputs:
```

```
    - x: input tensor (B, seq_len, input_size)
```

```
    Returns:
```

```
    - all_h: tensor of size (B, seq_len, hidden_size) containing hidden states  
      produced for each timestep
```

```
    - last_h: hidden state from the last timestep (B, hidden_size)
```

```
    """
```

```
    h_list = [] # List to store the hidden states [h_1, ... h_T]
```

```
#####  
#####
```

```
    # TODO: Implement the RNN forward step
```

```
    #
```

```
    # 1. Initialize h0 with zeros
```

```
    #
```

```
    # 2. Roll out the RNN over the sequence, storing hidden states in h_list
```

```
    #
```

```
    # 3. Return the appropriate outputs
```

```
    #
```

```
#####  
#####
```

```
#####  
#####
```

```
    #
```

```
    END OF YOUR CODE
```

```
    #
```

```
#####  
#####
```

```
    # h_list should now contain all hidden states, each of size (B, hidden_size)
```

```
    # We will store the hidden states so we can analyze their gradients later
```

```

self.store_h_for_grad(h_list)
all_h = th.stack(h_list, dim=1)
return all_h, last_h

```

```

def store_h_for_grad(self, h_list):
    """
    Store input list and allow gradient computation for all list elements
    """
    for h in h_list:
        h.retain_grad()
    self.h_list = h_list

```

ChatGPT

Here's a minimal vanilla RNN layer that follows the update

$$h_0 = \mathbf{0}, \quad h_t = \phi(\mathbf{x}_t W_{xh} + h_{t-1} W_{hh} + \mathbf{b})$$

with shapes $(\mathbf{x}_t \in \mathbb{R}^{B \times D}), (W_{xh} \in \mathbb{R}^{D \times H}), (W_{hh} \in \mathbb{R}^{H \times H}), (\mathbf{b} \in \mathbb{R}^H), (\phi = \text{nonlinearity})$ (default `torch.tanh`). I use two `nn.Linear` layers and keep a single bias on the input→hidden projection as suggested.

```

```python
class RNNLayer(nn.Module):
 def __init__(self, input_size, hidden_size, nonlinearity=th.tanh):
 """
 Initialize a single RNN layer.
 """
 super().__init__()
 self.input_size = input_size
 self.hidden_size = hidden_size
 self.nonlinearity = nonlinearity

```

```
#####
#####
TODO: Initialize any parameters your class needs.

#####
#####
Use two linear maps: x_t -> H (with bias), h_{t-1} -> H (no bias).
self.in2h = nn.Linear(input_size, hidden_size, bias=True)
self.h2h = nn.Linear(hidden_size, hidden_size, bias=False)

#####
#####
END OF YOUR CODE

#####
#####

def forward(self, x):
 """
 RNN forward pass

 Inputs:
 - x: input tensor (B, seq_len, input_size)

 Returns:
 - all_h: tensor of size (B, seq_len, hidden_size) containing hidden states
 produced for each timestep
 - last_h: hidden state from the last timestep (B, hidden_size)
 """
 h_list = [] # List to store the hidden states [h_1, ... h_T]

#####
#####
TODO: Implement the RNN forward step
1. Initialize h0 with zeros
2. Roll out the RNN over the sequence, storing hidden states in h_list
```



# 3. Return the appropriate outputs #

```

#####
```

```
B, T, _ = x.shape
h0: zeros matching input device/dtype
h_t = th.zeros(B, self.hidden_size, device=x.device, dtype=x.dtype)
```

```
Rollout over time
for t in range(T):
 x_t = x[:, t, :]
 # h_t = nonlinearity(x_t W_xh + h_{t-1} W_hh + b)
 h_t = self.nonlinearity(self.in2h(x_t) + self.h2h(h_t))
 h_list.append(h_t)
```

```
last_h = h_list[-1] if h_list else h_t # (B, H)
```

```

#####
```

```
END OF YOUR CODE
```

```

#####
```

```
h_list should now contain all hidden states, each of size (B, hidden_size)
We will store the hidden states so we can analyze their gradients later
self.store_h_for_grad(h_list)
all_h = th.stack(h_list, dim=1) # (B, T, H)
return all_h, last_h
```

```
def store_h_for_grad(self, h_list):
 """
 Store input list and allow gradient computation for all list elements
 """
 for h in h_list:
 h.retain_grad()
 self.h_list = h_list
```

```
...
```

**\*\*Notes/Justification (brief):\*\***

- Only one bias term is used (on `in2h`), complying with the tip.
- `h0` is zero-initialized per the instruction; roll-out is explicit over `seq\_len`.
- Shapes are preserved: each `h\_t` is `(B, H)`; stacking along time yields `(B, T, H)`.

**user**

# 1.B Implementing a RNN regression model.

```
class RecurrentRegressionModel(nn.Module):
 def __init__(self, recurrent_net, output_dim=1):
```

```
 """
```

```
 Initialize a simple RNN regression model
```

```
 Inputs:
```

- recurrent\_net: an RNN or LSTM (single or multi layer)
- output\_dim: feature dimension of the output

```
 """
```

```
 super().__init__()
```

```
 self.recurrent_net = recurrent_net
```

```
 self.output_dim = output_dim
```

```
#####
```

```
#####
```

```
 # TODO: Initialize any parameters you need #
```

```
 # HINT: use recurrent_net.hidden_size to find the hidden state size #
```

```
#####
```

```
#####
```

```
#####
```

```
#####
```

```
 # END OF YOUR CODE #
```

```

#####
```

```
def forward(self, x):
```

```
 """
```

```
 Forward pass
```

```
 Inputs:
```

```
 - x: input tensor (B, seq_len, input_size)
```

```
 Returns:
```

```
 - out: predictions of shape (B, seq_len, self.output_dim).
```

```
 - all_h: tensor of size (B, seq_len, hidden_size) containing hidden states
 produced for each timestep.
```

```
 """
```

```

#####
```

```
 # TODO: Implement the forward step.
```

```
 #
```

```

#####
```

```

#####
```

```
 #
```

```
 END OF YOUR CODE
```

```
 #
```

```

#####
```

```
 return out, all_h
```

**ChatGPT**



Here's a minimal seq-to-seq regression head on top of any RNN/LSTM-like module (uses `recurrent\_net.hidden\_size` and assumes `batch\_first=True`). It handles both our custom `RNNLayer` (which returns `(all\_h, last\_h)`) and PyTorch RNN/LSTM (which return `(output, (h\_n, c\_n))` or just `output`).

```
```python
class RecurrentRegressionModel(nn.Module):
    def __init__(self, recurrent_net, output_dim=1):
        """
        Initialize a simple RNN regression model
        """
        super().__init__()
        self.recurrent_net = recurrent_net
        self.output_dim = output_dim

#####
#####
# TODO: Initialize any parameters you need #
# HINT: use recurrent_net.hidden_size to find the hidden state size #
#####
#####
self.hidden_size = recurrent_net.hidden_size
# Linear readout applied at every timestep:  $y_t = h_t W + b$ 
self.readout = nn.Linear(self.hidden_size, output_dim, bias=True)

#####
#####
#                               END OF YOUR CODE                               #
#####
#####

def forward(self, x):
    """
    Forward pass

```

Inputs:

- x: input tensor (B, seq_len, input_size)

Returns:

- out: predictions of shape (B, seq_len, self.output_dim).

- all_h: tensor of size (B, seq_len, hidden_size) containing hidden states produced for each timestep.

"""

```
#####
```

```
#####
```

```
# TODO: Implement the forward step.
```

```
#
```

```
#####
```

```
#####
```

```
    r = self.recurrent_net(x)
```

```
    # Support either:
```

```
    # - custom RNNLayer: (all_h, last_h)
```

```
    # - PyTorch RNN/LSTM with batch_first=True: (output, (h_n, c_n)) or output  
    all_h = r[0] if isinstance(r, tuple) else r # (B, T, H)
```

```
    out = self.readout(all_h) # (B, T, output_dim)
```

```
#####
```

```
#####
```

```
#                      END OF YOUR CODE
```

```
#
```

```
#####
```

```
#####
```

```
    return out, all_h
```

```
...
```

****Brief justification:**** We compute hidden states $(h_t \in \mathbb{R}^H)$ from the recurrent net, then apply a shared affine map per timestep,

$\hat{y}_t = h_t W + b, \quad W \in \mathbb{R}^{H \times \text{output_dim}},$

$b \in \mathbb{R}^{\text{output_dim}},$

yielding predictions of shape $((B, T, \text{output_dim}))$. Assumes
`batch\_first=True` for PyTorch RNN/LSTM so shapes match the docstring.

user

1.C.ii Implement the loss function

```
def loss_fn(pred, y, last_timestep_only=False):
```

```
    """
```

```
    Inputs:
```

- pred: model predictions of size (batch, seq_len, 1)
- y: targets of size (batch, seq_len, 1)
- last_timestep_only: boolean indicating whether to compute loss for all timesteps or only the last

```
    Returns:
```

- loss: scalar MSE loss between pred and true labels

```
    """
```

```
#####  
#####
```

```
    # TODO: implement the loss (HINT: look for pytorch's MSELoss function)    #
```

```
#####  
#####
```

```
#####  
#####
```

```
    #                                END OF YOUR CODE                                #
```

```
#####  
#####
```

```
    return loss
```

ChatGPT

```
```python
```



```
def loss_fn(pred, y, last_timestep_only=False):
 """
 Inputs:
 - pred: model predictions of size (batch, seq_len, 1)
 - y: targets of size (batch, seq_len, 1)
 - last_timestep_only: if True, compute loss only at the last timestep

 Returns:
 - loss: scalar MSE loss between pred and true labels
 """
 # If requested, only compare the last timestep (index -1)
 if last_timestep_only:
 pred = pred[:, -1, :] # (B, 1)
 y = y[:, -1, :] # (B, 1)

 loss = nn.MSELoss(reduction='mean')(pred, y)
 return loss
...
```

## **user**

### **# 1.D: Analyzing RNN Gradients**

```
def biggest_eig_magnitude(matrix):
 """
 Inputs: a square matrix
 Returns: the scalar magnitude of the largest eigenvalue
 """
 h, w = matrix.shape
 assert h == w, f'Matrix has shape {matrix.shape}, but eigenvalues can only be
 computed for square matrices'
 eigs = th.linalg.eigvals(matrix)
 eig_magnitude = eigs.abs()
 eigs_sorted = sorted([i.item() for i in eig_magnitude], reverse=True)
 first_eig_magnitude = eigs_sorted[0]
 return first_eig_magnitude
```

```
class GradientVisualizer:
```

```
def __init__(self, rnn, last_timestep_only):
```

```
 """
```

```
 Inputs:
```

- rnn: rnn module
- last\_timestep\_only: boolean indicating whether to compute loss for all timesteps or only the last

```
 Returns:
```

- loss: scalar MSE loss between pred and true labels

```
 """
```

```
 self.rnn = rnn
```

```
 self.last_timestep_only = last_timestep_only
```

```
 self.model = RecurrentRegressionModel(rnn)
```

```
 self.original_weights = copy.deepcopy(rnn.state_dict())
```

```
 # Generate a single batch to be used repeatedly
```

```
 self.x, self.y = generate_batch(seq_len=10)
```

```
 print(f'Data point: x={np.round(self.x[0, :, 0].detach().cpu().numpy(), 2)},
y={np.round(self.y.squeeze().detach().cpu().numpy(), 2)}')
```

```
def plot_visuals(self):
```

```
 """ Generate plots which will be updated in realtime. """
```

```
 fig, (ax1, ax2) = plt.subplots(1, 2)
```

```
 ax1.set_title('RNN Outputs')
```

```
 ax1.set_xlabel('Unroll Timestep')
```

```
 ax1.set_ylabel('Hidden State Norm')
```

```
 ax1.set_ylim(-1, 5)
```

```
 plt_1 = ax1.plot(np.arange(1, 11), np.zeros(10) + 1) # placeholder vals
```

```
 plt_1 = plt_1[0]
```

```
 ax2.set_title('Gradients')
```

```
 ax2.set_xlabel('Unroll Timestep')
```

```
 ax2.set_ylabel('RNN dLoss/d a_t Gradient Magitude')
```

```
 ax2.set_ylim((10**-6, 1e5))
```

```

 ax2.set_yscale('log')
 # X-axis labels are reversed since the gradient flow is from later layers to
earlier layers
 ax2.set_xticks(np.arange(10), np.arange(10, 0, -1))
 plt_2 = ax2.plot(np.arange(10), np.arange(10) + 1) # placeholder vals
 plt_2 = plt_2[0]
 self.fig = fig
 self.plots = [plt_1, plt_2]
 return plt_1, plt_2, fig

Main update function for interactive plot
def update_plots(self, weight_val=0, bias_val=0):
 # Scale the original RNN weights by a constant
 w_dict = copy.deepcopy(self.original_weights)

#####
#####
 # TODO: Scale all W matrixes by weight_val, and all bias matrices by
bias_val#
 # If you're using PyTorch nn.Linear layers, you don't need to modify the code#
 # provided, but if you're using custom layers, modify this block. #

#####
#####

#####
#####

END OF YOUR CODE

#####
#####
 self.rnn.load_state_dict(w_dict)

Don't compute for LSTMs, which don't have behavior dependent on a single
eigenvalue
 if isinstance(self.rnn, RNNLayer):

```



```
#####
#####
TODO: Set W = the weight which most affects exploding/vanishing
gradients #
Hint: Call module.weight or module.bias on the module you want to use
#
If you used a single Linear layer, slice a square matrix from it.

#####
#####

#####
#####
END OF YOUR CODE

#####
#####
 biggest_eig = biggest_eig_magnitude(W)
 print(f' Biggest eigenvalue magnitude: {biggest_eig:.3}')

Run model
pred, h = self.model(self.x)
loss = loss_fn(pred, self.y, self.last_timestep_only)
n_steps = len(h[0])

plt_1, plt_2 = self.plots

Plot the hidden state magnitude
max_h = th.linalg.norm(h[0], dim=-1).detach().cpu().numpy()
print('Max H', ' '.join([f'{num:.3}' for num in max_h]))
plt_1.set_data(np.arange(1, n_steps + 1), np.array(max_h))
Compute the gradient for the loss wrt the stored hidden states
Gradients are plotted backward since we go from later layers to earlier
grads = [th.linalg.norm(num).item() for num in th.autograd.grad(loss,
self.rnn.h_list)][::-1]
print('gradients d Loss/d h_t', ' '.join([f'{num:.3}' for num in grads]))
```

```

Add 1e-6 since it throws an error for gradients near 0
plt_2.set_data(np.arange(n_steps), np.array(grads) + 1e-6)
self.fig.canvas.draw_idle()

def create_visualization(self):
 # Include sliders for relevant quantities
 self.plot_visuals()
 ip = interactive(self.update_plots,
 weight_val=widgets.FloatSlider(value=0, min=-5, max=5, step=.05,
description="weight_scale", layout=Layout(width='100%')),
 bias_val=widgets.FloatSlider(value=0, min=-5, max=5, step=.05,
description="bias_scale", layout=Layout(width='100%')),
)

 return ip

```

## ChatGPT

Filled in the two TODOs with the minimal, tip-aligned logic:

- Scale **all weights** by  $(10^{\text{weight\_val}})$  and **all biases** by  $(10^{\text{bias\_val}})$  (so slider `0`  $\Rightarrow$  no change).
- For the eigenvalue readout on a vanilla RNN, use the **hidden-to-hidden** matrix  $(W_{hh})$  (the one that drives exploding/vanishing via its spectral radius).

```

```python
# In GradientVisualizer.update_plots
def update_plots(self, weight_val=0, bias_val=0):
    # Scale the original RNN weights by a constant
    w_dict = copy.deepcopy(self.original_weights)

#####
#####
    # TODO: Scale all W matrixes by weight_val, and all bias matrixes by bias_val#
    # If you're using PyTorch nn.Linear layers, you don't need to modify the code#
    # provided, but if you're using custom layers, modify this block.      #

```



```
#####
#####
w_scale = 10.0 ** float(weight_val)
b_scale = 10.0 ** float(bias_val)
for k, v in w_dict.items():
    if 'weight' in k:
        w_dict[k] = v * w_scale
    elif 'bias' in k:
        w_dict[k] = v * b_scale

#####
#####
#                               END OF YOUR CODE                               #

#####
#####
self.rnn.load_state_dict(w_dict)

# Don't compute for LSTMs, which don't have behavior dependent on a single
eigenvalue
if isinstance(self.rnn, RNNLayer):

#####
#####
# TODO: Set W = the weight which most affects exploding/vanishing gradients
#
# Hint: Call module.weight or module.bias on the module you want to use      #
# If you used a single Linear layer, slice a square matrix from it.          #

#####
#####
# For a vanilla RNN:  $h_t = \phi(W_{xh} x_t + W_{hh} h_{t-1} + b)$ . Grad flow mainly
# governed by spectral radius  $\rho(W_{hh})$ , so select the recurrent matrix:
W = self.rnn.h2h.weight.detach()
```

```
#####
#####
```

```
#                      END OF YOUR CODE                      #
```

```
#####
#####
```

```
    biggest_eig = biggest_eig_magnitude(W)
    print(f' Biggest eigenvalue magnitude: {biggest_eig:.3}')
```

```
# (rest unchanged) ...
```

```
pred, h = self.model(self.x)
```

```
loss = loss_fn(pred, self.y, self.last_timestep_only)
```

```
n_steps = len(h[0])
```

```
plt_1, plt_2 = self.plots
```

```
max_h = th.linalg.norm(h[0], dim=-1).detach().cpu().numpy()
```

```
print('Max H', ' '.join([f'{num:.3}' for num in max_h]))
```

```
plt_1.set_data(np.arange(1, n_steps + 1), np.array(max_h))
```

```
grads = [th.linalg.norm(num).item() for num in th.autograd.grad(loss,
self.rnn.h_list)][:-1]
```

```
print('gradients d Loss/d h_t', ' '.join([f'{num:.3}' for num in grads]))
```

```
plt_2.set_data(np.arange(n_steps), np.array(grads) + 1e-6)
```

```
self.fig.canvas.draw_idle()
```

```
...
```

```
### Brief reasoning (gradient behavior)
```

For $(h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b))$ with pre-activation

$(a_t = W_{xh}x_t + W_{hh}h_{t-1} + b)$,

∇

$\frac{\partial h_t}{\partial$

$h_{t-1} = \text{operatorname{diag}}(\phi'(a_t))W_{hh}$.

∇

Across (τ) steps,

∇

$$\frac{\partial h_t}{\partial h_{t-\tau}} = \prod_{k=t-\tau+1}^t \operatorname{diag}(\phi'(a_k) W_{hh}).$$

Hence $\frac{\partial h_t}{\partial h_{t-\tau}}$ is governed by $\prod_k \operatorname{diag}(\phi'(a_k)) \cdot W_{hh}$, and in practice by the spectral radius $\rho(W_{hh})$: $\rho(W_{hh}) > 1 \Rightarrow \text{exploding}$, $\rho(W_{hh}) < 1 \Rightarrow \text{vanishing}$. That's why we report the largest $|\lambda|$ of (W_{hh}) .

user

1.K.i: Implementing multi-layer models

```
class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers):
        """
        Initialize a multilayer RNN

        Inputs:
        - input_size: Data input feature dimension
        - hidden_size: hidden state size (also the output feature dimension)
        - num_layers: number of layers
        """
        super().__init__()
        assert num_layers >= 1
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers

#####
#####
# TODO: Initialize any parameters your class needs.          #
# Consider using nn.ModuleList or nn.ModuleDict.             #
#####
#####
```

```
#####  
#####
```

```
#                      END OF YOUR CODE                      #
```

```
#####  
#####
```

```
def forward(self, x):
```

```
    """
```

```
    Multilayer RNN forward pass
```

```
    Inputs:
```

```
    - x: input tensor (B, seq_len, input_size)
```

```
    Returns:
```

```
    - last_layer_h: tensor of size (B, seq_len, hidden_size) containing the  
      outputs produced for each timestep from the last layer
```

```
    - last_step_h: all hidden states from the last step (num_layers, B, hidden_size)
```

```
    """
```

```
#####  
#####
```

```
    # TODO: Implement the RNN forward step                      #
```

```
#####  
#####
```

```
#####  
#####
```

```
#                      END OF YOUR CODE                      #
```

```
#####  
#####
```

```
    return last_layer_h, last_step_h
```

ChatGPT

```

```python
class RNN(nn.Module):
 def __init__(self, input_size, hidden_size, num_layers):
 """
 Initialize a multilayer RNN
 """
 super().__init__()
 assert num_layers >= 1
 self.input_size = input_size
 self.hidden_size = hidden_size
 self.num_layers = num_layers

#####
#####
TODO: Initialize any parameters your class needs.
Consider using nn.ModuleList or nn.ModuleDict.

#####
#####
 layers = []
 in_size = input_size
 for _ in range(num_layers):
 layers.append(RNNLayer(in_size, hidden_size)) # each layer outputs (B,T,H)
 in_size = hidden_size # next layer consumes H
 self.layers = nn.ModuleList(layers)

#####
#####
END OF YOUR CODE

#####
#####

def forward(self, x):
 """
 Multilayer RNN forward pass

```



Inputs:

- x: input tensor (B, seq\_len, input\_size)

Returns:

- last\_layer\_h: tensor of size (B, seq\_len, hidden\_size) containing the outputs produced for each timestep from the last layer  
- last\_step\_h: all hidden states from the last step (num\_layers, B, hidden\_size)  
""

```

#####
```

```
TODO: Implement the RNN forward step
```

```

#####
```

```
last_states = []
out = x
for layer in self.layers:
 all_h, last_h = layer(out) # all_h: (B,T,H), last_h: (B,H)
 last_states.append(last_h)
 out = all_h # feed sequence to next layer
```

```
last_layer_h = out # (B,T,H) from final layer
last_step_h = th.stack(last_states, dim=0) # (L,B,H) last step per layer
```

```
Expose last layer's h_t list for gradient probes used elsewhere
self.h_list = self.layers[-1].h_list
```

```

#####
```

```
END OF YOUR CODE
```

```

#####
```

```
 return last_layer_h, last_step_h
...
```

**\*\*Notes (brief):\*\***

- Stack `RNNLayer`s in an `nn.ModuleList`; first consumes `input\_size`, others consume `hidden\_size`.
- For gradients visualization used elsewhere, we forward the last layer's `h\_list` via `self.h\_list`.
- Shapes match the spec: final sequence outputs `(B,T,H)` and last-step states per layer `(L,B,H)`.

## **user**

The main objective of this task is to predict the probability of a given class given a last name, represented as

$$\Pr ( y \mid x_1, x_2, x_3, \dots, x_i), \Pr$$

where  $y$  is the category label and each  $x_i$  is a character in the last name. Building a basic character-level NLP model has the advantage of understanding how the preprocessing works at a granular level. The character-level network reads words as a sequence of characters, producing a prediction and "hidden state" at each step by feeding its previous hidden state into the next step. The final prediction corresponds to the class to which the word belongs.

All models in PyTorch inherit from the `nn.Module` subclass. In this assignment, you will **implement a custom model named `RecurrentClassifier`** that runs either `[nn.RNN]`(<https://pytorch.org/docs/stable/generated/torch.nn.RNN.html>) or `[nn.LSTM]`(<https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>) and define its forward function. The implementation of LSTMs is *optional*.

The forward pass of the model can be visualized with the following diagram:

...

[Embedding] -> [RNN Stack] -> [Extract Last Position] -> [Classifier]

...

- **\*\*Embedding:\*\*** This component maps each input word (integer) to a vector of real numbers.
  - Input: `[batch\_size, seq\_len]`

- Output: `[batch\_size, seq\_len, rnn\_size]`
- **\*\*RNN Stack:\*\*** This component consists of one or more RNN layers, which process the input sequence of vectors from the Embedding component.
  - Input: `[batch\_size, seq\_len, rnn\_size]`
  - Output: `[batch\_size, seq\_len, rnn\_size]`
- **\*\*Extract Last Position:\*\*** The RNN Stack component returns a sequence of vectors for each input example. However, for classification purposes, we only need a single vector that captures the full information of the input example. Since the RNN is left-to-right by default, the output state vector at the last position contains the full information of the input example. Therefore, for the  $i$ -th input example, we extract the output state vector at the last *\*non-pad\** position, which is indicated by `last_pos[i]`.
  - Input: `[batch\_size, seq\_len, rnn\_size]`
  - Output: `[batch\_size, rnn\_size]`
- **\*\*Classifier:\*\*** This component is a fully-connected layer that maps the output vectors extracted in the previous step to logits (scores before softmax), which can be used to make predictions about the language of origin for each input example.
  - Input: `[batch\_size, rnn\_size]`
  - Output: `[batch\_size, n\_categories]`

These documents would be helpful in this part:

- <https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>
- <https://pytorch.org/docs/stable/generated/torch.nn.RNN.html>
- <https://pytorch.org/docs/stable/generated/torch.gather.html>
- <https://pytorch.org/docs/stable/generated/torch.Tensor.expand.html>
- <https://pytorch.org/docs/stable/generated/torch.Tensor.view.html>

```
class RecurrentClassifier(nn.Module):
```

```
 def __init__(
 self,
 vocab_size: int,
 rnn_size: int,
```



```
n_categories: int,
num_layers: int = 1,
dropout: float = 0.0,
model_type: str = 'lstm'
):
 super().__init__()
 self.rnn_size = rnn_size
 self.model_type = model_type
```

```
#####
#####
```

```
TODO: Create an embedding layer of shape [vocab_size, rnn_size]
#
Hint: Use nn.Embedding
https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html
It will map each word into a vector of shape [rnn_size]
```

```
#####
#####
```

```
#####
#####
```

```
#####
#####
```

```
TODO: Create a RNN stack with `num_layers` layers with tanh
nonlinearity. Between each layers, there is a dropout of
`dropout`. Implement it with a *single* call to `torch.nn` APIs
#
Hint: See documentations at
https://pytorch.org/docs/stable/generated/torch.nn.RNN.html
Set the arguments to call `nn.RNN` such that:
- The shape of the input is [batch_size, seq_len, rnn_size]
- The shape of the output should be [batch_size, seq_len, rnn_size]
Make sure that the dimension ordering is correct. One of the argument
```

```

in the constructor of `nn.RNN` (or `nn.LSTM`) is helpful here
#
Optional: Implement one LSTM layer when `model_type` is `lstm`

#####
#####

#####
#####

#####
#####
TODO: Implement one dropout layer and the fully-connected classifier
layer
#
Hint: We add a dropout layer because neither nn.RNN nor nn.LSTM
implements dropout after the last layer in the stack.
Since the input to the classifier is the output of the last position
of the RNN's final layer, it has a shape of [batch_size, rnn_size].
The expected output should be logits, which correspond to scores
before applying softmax, and should have a shape of
[batch_size, n_categories].

#####
#####

#####
#####

def forward(self, x: torch.Tensor, last_pos: torch.Tensor) -> torch.Tensor:
 """
 x: integer tensor of shape [batch_size, seq_len]
 last_pos: integer tensor of shape [batch_size]

```

The input tensor `x` is composed of a batch of sequences, where each sequence contains indices corresponding to characters. As sequences

within the same batch may have different lengths, shorter sequences are padded on the right side to match the maximum sequence length of the batch, which is represented by `seq\_len`.

Additionally, the `last\_pos` tensor records the position of the last character in each sequence. For instance, the first sequence in the batch can be represented as `[x[0, 0], x[0, 1], ..., x[0, last\_pos[0]]`.

`last\_pos` is useful when extracting the output state associated with each sequence from the RNNs.

```
"""
```

```
embeds = self.embedding(x)
if self.model_type == 'lstm':
 rnn_out, _ = self.lstm(embeds)
else:
 rnn_out, _ = self.rnn(embeds)
```

```
#####
#####
```

```
TODO: Retrieve the output state associated with each sequence
#
Hints:
- The output state of all positions is returned by the RNN stack,
but we only need the state in the last position for classification
- The shape of `rnn_out` is [batch_size, seq_len, rnn_size]
- The expected shape of `out` is [batch_size, rnn_size]
- For the i-th sequence, we have out[i] == rnn_out[i, last_pos[i]]
- Try to condense your code into a single line, without using any
loops. However, if you find it too challenging to do so, you may use
a single layer of for-loop.
```

```
#####
#####
```

```
#####
#####
```

```
out = self.drop(out)
logits = self.output(out)
return logits
```

## ChatGPT

Here's a minimal, tip-aligned implementation. It uses `nn.Embedding`, an RNN stack (`nn.RNN` by default, optional `nn.LSTM`), gathers the last non-pad position per example in one line, then applies dropout and a linear classifier.

```
```python
class RecurrentClassifier(nn.Module):
    def __init__(
        self,
        vocab_size: int,
        rnn_size: int,
        n_categories: int,
        num_layers: int = 1,
        dropout: float = 0.0,
        model_type: str = 'lstm'
    ):
        super().__init__()
        self.rnn_size = rnn_size
        self.model_type = model_type

#####
#####
        # TODO: Create an embedding layer of shape [vocab_size, rnn_size]

#####
#####
        self.embedding = nn.Embedding(vocab_size, rnn_size)

#####
#####
```



```
#####  
#####
```

```
# TODO: Create a RNN stack with `num_layers` layers with tanh  
#     nonlinearity, batch_first=True, and dropout between layers.  
# Optional LSTM when `model_type == 'lstm'`.
```

```
#####  
#####
```

```
if model_type == 'lstm':  
    self.lstm = nn.LSTM(  
        input_size=rnn_size,  
        hidden_size=rnn_size,  
        num_layers=num_layers,  
        dropout=dropout if num_layers > 1 else 0.0,  
        batch_first=True,  
    )  
else:  
    self.rnn = nn.RNN(  
        input_size=rnn_size,  
        hidden_size=rnn_size,  
        num_layers=num_layers,  
        nonlinearity='tanh',  
        dropout=dropout if num_layers > 1 else 0.0,  
        batch_first=True,  
    )
```

```
#####  
#####
```

```
#####  
#####
```

```
# TODO: Implement one dropout layer and the fully-connected classifier
```

```
#####
#####
    self.drop = nn.Dropout(p=dropout)
    self.output = nn.Linear(rnn_size, n_categories)

#####
#####

def forward(self, x: torch.Tensor, last_pos: torch.Tensor) -> torch.Tensor:
    """
    x: [batch_size, seq_len] (integer indices)
    last_pos: [batch_size] (position of last non-pad per sequence)
    """
    embeds = self.embedding(x) # [B, T, H]
    if self.model_type == 'lstm':
        rnn_out, _ = self.lstm(embeds) # [B, T, H]
    else:
        rnn_out, _ = self.rnn(embeds) # [B, T, H]

#####
#####
    # TODO: Retrieve the output state at the last position for each sample.
    # One-line gather along the time dimension.

#####
#####
    out = rnn_out.gather(
        dim=1,
        index=last_pos.to(rnn_out.device).view(-1, 1, 1).expand(-1, 1,
self.rnn_size)
    ).squeeze(1) # [B, H]

#####
#####
```

```

        out = self.drop(out)
        logits = self.output(out) # [B, n_categories]
        return logits
    ...

```

Why this works (concise)

- **Embedding:** maps tokens $\rightarrow$ vectors: $[B, T] \rightarrow [B, T, H]$.
- **RNN stack:** preserves time dimension: $[B, T, H] \rightarrow [B, T, H]$.
- **Extract last position:** `gather` indexes per-row at `last_pos[i]` across the sequence dimension (dim=1), then squeezes to $[B, H]$.
- **Classifier:** affine map to logits: $[B, H] \rightarrow [B, C]$. Dropout is applied once after extraction to cover the “no last-layer dropout” gap in PyTorch RNN/LSTM.

user

****Implement the function to encode a letter to an integer:****

```
def letterToIndex(letter):
```

```
#####
#####
```

```

    # TODO: implement the function to map a letter (a character) into its index
    #     in `all_letters`
    #
    # e.g. letterToIndex("a") == 0
    # Don't worry about efficiency here

```

```
#####
#####
```

```
#####
#####
```

```
assert letterToIndex("a") == 0
```

```
assert letterToIndex("") == 56
```

ChatGPT

```
```python
def letterToIndex(letter):

#####
#####
 # Map a single character to its index in `all_letters`

#####
#####
 return all_letters.index(letter)

```
```

user

****Adjust the hyperparameters listed below**** to train an RNN with a minimum evaluation accuracy of 80% after 20 epochs.

```
criterion = nn.CrossEntropyLoss()

# The build_batch function outputs numpy, but our model is built in pytorch,
# so we need to convert numpy to pytorch with the correct types.
batch_to_torch = lambda b_in,b_target,b_mask: (torch.tensor(b_in).long(),
                                                torch.tensor(b_target).long(),
                                                torch.tensor(b_mask).long())

#####
#####
# TODO: Tune these hyperparameters for a better performance
#####
#####
#####
#####

# Do not change the number of epochs
```




```
epochs = 20
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("You are using", device, "for training")
```

```
list_to_device = lambda th_obj: [tensor.to(device) for tensor in th_obj]
```

ChatGPT

```
```python
#####
#####
TODO: Tune these hyperparameters for a better performance
Target: ≥80% eval accuracy by epoch 20 (character-level surname classifier)
#####
#####
Model
model_type = 'lstm' # 'lstm' (recommended) or 'rnn'
rnn_size = 256 # embedding dim = hidden size
num_layers = 2
dropout = 0.30

Optimization
batch_size = 256
learning_rate= 3e-3
weight_decay = 1e-5 # mild L2 helps generalization
betas = (0.9, 0.99) # if using Adam

Training hygiene
grad_clip = 5.0 # clip global norm to curb exploding grads

(Optional) LR schedule if your loop supports it
sched_step = 8 # decay LR at epochs 8 and 14
sched_gamma = 0.5
#####
#####
```
```

user

Question

Although the neural network you have trained is intended to predict the language of origin for a given last name, it could potentially be misused. **In what ways do you think this could be problematic in real-world applications?** Include your answer in your submission of the written assignment.

ChatGPT

Here are concrete ways a surname→language-of-origin classifier can be harmful or misused:

1) **Proxy discrimination / disparate impact.**

Organizations could use the model to infer protected attributes (ethnicity, national origin, religion by proxy) and make decisions in hiring, housing, lending, insurance, or content moderation. Even if “language of origin” is the stated target, it functions as a proxy and can violate anti-discrimination norms and laws.

2) **Surveillance and profiling.**

Law enforcement, border control, or schools could profile individuals or communities, prioritize “extra screening,” or create watchlists based on predicted origin—amplifying existing biases and chilling lawful behavior.

3) **Targeted manipulation and harassment.**

Advertisers, political groups, or malicious actors could micro-target communities for disinformation, predatory services, scams, or hate campaigns once the model groups users by inferred origin.

4) **Privacy violations and de-anonymization.**

Inferring sensitive attributes from a last name without consent can breach privacy expectations. Combined with data-broker records, model outputs can help re-identify individuals or reveal information they never chose to disclose.

5) **Misclassification harms (representational harm).**

Names change through marriage, adoption, diaspora, and transliteration. Many people have multi-ethnic backgrounds. Wrong labels can erase identities, cause offense, or lead to inaccurate downstream records (“official” systems treating the model’s guess as truth).

6) **Bias amplification and uneven error rates.**

Training data are rarely balanced across cultures, scripts, and diacritics. Minority or under-represented groups tend to get higher error rates, which—if used in automated pipelines—produce systematic disadvantages.

7) **Overconfidence and misuse of probabilities.**

Stakeholders may treat softmax scores as ground truth. Without calibration and abstention, low-confidence guesses can still drive consequential actions (base-rate fallacy, thresholding choices) and create false certainty.

8) **Feedback loops.**

If predictions are used to label—or filter—future data (e.g., “only keep names predicted as X”), the training distribution drifts toward stereotypes, worsening bias over time.

9) **Security / model-privacy risks.**

Adversaries might mount membership-inference or model-inversion attacks to learn whether specific names were in the training set or to extract sensitive patterns from the model.

Practical safeguards (brief):

- Do **not** use for individual-level or high-stakes decisions; require human review.
- Provide an **“Unknown/Abstain”** option and set conservative confidence thresholds with **calibrated** probabilities.
- Publish a **model card** with data sources, limitations, and bias audits; evaluate per-group error rates.
- Limit access, rate-limit queries, log misuse, and avoid storing personally identifying inputs.

- Obtain consent for any attribute inference; avoid combining outputs with data-broker PII.
- Regularly retrain and re-audit with diverse, diacritic-preserving data; include fairness tests.

Even with strong accuracy, the ethical and legal risks above mean deployment should be narrowly scoped, transparent, and accompanied by explicit guardrails.

user

Vanilla Autoencoder

In this section, you will be implementing a vanilla autoencoder, which comprises of an encoder and a decoder, both of which are fully connected neural networks. The input $x \in \mathbb{R}^d$ is mapped to a latent representation z by the encoder, which is then mapped back to x' . During training, the mean squared error between x and x' is minimized using the following formula:

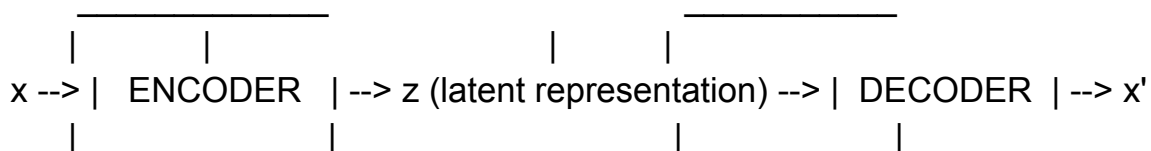
$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^d |x_{i,j} - x'_{i,j}|^2$$

Here, n is the number of samples in the dataset, d is the dimensionality of each sample, $x_{i,j}$ is the j -th feature of the i -th sample, and $x'_{i,j}$ is the predicted value of the j -th feature of the i -th sample.

```
class Autoencoder(nn.Module):
```

```
    """
```

```
    Autoencoder defines a general class of NN architectures
```



The `Autoencoder` class is a neural network architecture consisting of an encoder and a decoder, each of which is a fully connected neural network.

The input `x` of size `input_size` is mapped to a latent representation `z`

by the encoder, which is then mapped back to x' by the decoder.

The architecture is defined by a list of hidden layer sizes for the encoder and decoder. The encoder and decoder are symmetric. The class provides methods for encoding, decoding, and computing the loss (mean squared error) between x and x' . A training step can be performed by calling the `train_step` method with an input tensor x and an optimizer.

```
"""
```

```
def __init__(self, input_size: int, hidden_sizes: List[int],
              activation_cls: nn.Module = nn.ReLU):
    super().__init__()
    self.input_size = input_size
    self.hidden_sizes = hidden_sizes
    self.activation_cls = activation_cls
    self.encoder = self._build_encoder()
    self.decoder = self._build_decoder()
```

```
def _build_encoder(self):
    layers = []
    prev_size = self.input_size
    for layer_id, size in enumerate(self.hidden_sizes):
        layers.append(nn.Linear(prev_size, size))
        if layer_id < len(self.hidden_sizes)-1:
            layers.append(self.activation_cls())
        prev_size = size
    return nn.Sequential(*layers)
```

```
def _build_decoder(self):
    layers = []
```

```
#####
```

```
#####
```

```
    # TODO: Implement the code to construct the decoder. The decoder should
    #       be symmetric to the encoder.
    #
    # Hint: Refer to the `_build_encoder` method above
```

```
#####
#####
    prev_size = self.hidden_sizes[-1]
    for size in reversed(self.hidden_sizes[:-1]):
        layers.append(nn.Linear(prev_size, size))
        layers.append(self.activation_cls())
        prev_size = size
    layers.append(nn.Linear(prev_size, self.input_size))

#####
#####
    return nn.Sequential(*layers)

def forward(self, x: torch.Tensor) -> torch.Tensor:

#####
#####
    # TODO: Implement the forward pass of the (vanilla) autoencoder
    #     according to the diagram and documents above
    #     The return value should be `x`

#####
#####
    z = self.encoder(x)
    x_hat = self.decoder(z)
    return x_hat

#####
#####

def get_loss(self, x):
    x_hat = self(x)
    return self.loss(x, x_hat)

def encode(self, x):
    return self.encoder(x)
```

```
def decode(self, z):
    return self.decoder(z)
```

```
def loss(self, x: torch.Tensor, x_hat: torch.Tensor) -> torch.Tensor:
```

```
#####
#####
```

```
    # TODO: Implement the loss function
```

```
#####
#####
```

```
    return F.mse_loss(x, x_hat, reduction="sum") / x.size(0)
```

```
#####
#####
```

```
def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:
```

```
    x_hat = self(x)
    loss = self.loss(x, x_hat)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

```
    return loss
```

ChatGPT

```
```python
```

```
class Autoencoder(nn.Module):
```

```
 def __init__(self, input_size: int, hidden_sizes: List[int],
 activation_cls: nn.Module = nn.ReLU):
```

```
 super().__init__()
```

```
 self.input_size = input_size
```

```
 self.hidden_sizes = hidden_sizes
```

```
 self.activation_cls = activation_cls
```

```
 self.encoder = self._build_encoder()
```

```
 self.decoder = self._build_decoder()
```



```
def _build_encoder(self):
 layers = []
 prev_size = self.input_size
 for layer_id, size in enumerate(self.hidden_sizes):
 layers.append(nn.Linear(prev_size, size))
 if layer_id < len(self.hidden_sizes)-1:
 layers.append(self.activation_cls())
 prev_size = size
 return nn.Sequential(*layers)
```

```
def _build_decoder(self):
 layers = []
```

```
#####
#####
```

```
 # Symmetric to the encoder: go from latent -> ... -> input_size,
 # add activations after intermediate layers, none on the final layer.
```

```
#####
#####
```

```
 prev_size = self.hidden_sizes[-1]
 for size in reversed(self.hidden_sizes[:-1]):
 layers.append(nn.Linear(prev_size, size))
 layers.append(self.activation_cls())
 prev_size = size
 layers.append(nn.Linear(prev_size, self.input_size))
```

```
#####
#####
```

```
 return nn.Sequential(*layers)
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
#####
#####
```

```
 # x -> encoder -> z -> decoder -> x_hat
```



```

#####
```

```
 z = self.encoder(x)
 x_hat = self.decoder(z)
 return x_hat
```

```

#####
```

```
def get_loss(self, x):
 x_hat = self(x)
 return self.loss(x, x_hat)
```

```
def encode(self, x):
 return self.encoder(x)
```

```
def decode(self, z):
 return self.decoder(z)
```

```
def loss(self, x: torch.Tensor, x_hat: torch.Tensor) -> torch.Tensor:
```

```

#####
```

```
 # Implements $(1/n) * \sum_i ||x_i - x'_i||^2$ (sum over features per
 # sample, then mean over batch). Equivalent to F.mse_loss with sum=batch.
```

```

#####
```

```
 return F.mse_loss(x, x_hat, reduction="sum") / x.size(0)
 # alternatively: return ((x - x_hat) ** 2).sum(dim=1).mean()
```

```

#####
```

```
def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:
 x_hat = self(x)
```

```

 loss = self.loss(x, x_hat)
 optimizer.zero_grad()
 loss.backward()
 optimizer.step()
 return loss

 ...

```

**user**

## Denoising Autoencoder

In this section, you will be implementing a denoising autoencoder, which inherits vanilla autoencoder you implemented before, but with an added noise reduction part. The input  $x \in \mathbb{R}^d$  should be corrupted with Gaussian noise during training, and then fed to the encoder to obtain the latent representation  $z$ . The decoder then maps the latent representation back to the original, noise-free input  $x'$ . During training, the mean squared error between  $x$  and  $x'$  is minimized, similar to the vanilla autoencoder.

```

class DenoisingAutoencoder(Autoencoder):
 def __init__(self, input_size: int, hidden_sizes: List[int],
 activation_cls: nn.Module = nn.ReLU, noise_std: float = 0.5):
 super().__init__(input_size, hidden_sizes, activation_cls)
 self.noise_std = noise_std

 def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:

#####
#####
 # TODO: Implement training step of the denoising autoencoder.
 #
 # Hint: Add a zero-mean i.i.d. gaussian noise of a standard deviation of
 # `noise_std` to the input of the encoder

#####
#####
 x_noisy = x + self.noise_std * torch.randn_like(x)

```

```
x_hat = self(x_noisy)
```

```

#####
```

```
 loss = self.loss(x, x_hat)
 optimizer.zero_grad()
 loss.backward()
 optimizer.step()
```

```
 return loss
```

## ChatGPT

```
```python
```

```
class DenoisingAutoencoder(Autoencoder):
```

```
    def __init__(self, input_size: int, hidden_sizes: List[int],  
                  activation_cls: nn.Module = nn.ReLU, noise_std: float = 0.5):  
        super().__init__(input_size, hidden_sizes, activation_cls)  
        self.noise_std = noise_std
```

```
    def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:
```

```
#####  
#####
```

```
        # Add zero-mean i.i.d. Gaussian noise to inputs, reconstruct the clean x,  
        # backprop MSE(x, x_hat), and update parameters.
```

```
#####  
#####
```

```
        self.train()  
        x_noisy = x + self.noise_std * torch.randn_like(x)  
        x_hat = self(x_noisy)  
        loss = self.loss(x, x_hat)  
        optimizer.zero_grad(set_to_none=True)  
        loss.backward()  
        optimizer.step()  
        return loss
```

...

user

Masked Autoencoder

In this section, you will be implementing a masked autoencoder, which is similar to the vanilla autoencoder, but with an added masking feature. During training, the input $x \in \mathbb{R}^d$ should be masked with some binary mask, which zeros-out some random features in the input. The masked input is then fed to the encoder to obtain the latent representation z , and the decoder maps the latent representation back to the original input x' . During training, the mean squared error between the unmasked part of x and the corresponding part of x' is minimized.

```
class MaskedAutoencoder(Autoencoder):
```

```
    def __init__(self, input_size: int, hidden_sizes: List[int],
                  activation_cls: nn.Module = nn.ReLU, mask_prob: float = 0.25):
        super().__init__(input_size, hidden_sizes, activation_cls)
        self.mask_prob = mask_prob
```

```
    def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:
```

```
#####
```

```
#####
```

```
    # TODO: Implement training step of the masked autoencoder.
```

```
    #
```

```
    # Hint: Generate a mask with i.i.d. probabilities of mask_prob for each
```

```
    # entry, and apply it to the input of the encoder, setting to zero
```

```
    # the entries where the mask is activated.
```

```
#####
```

```
#####
```

```
        mask = torch.rand(x.shape, device=x.device) > self.mask_prob
```

```
        x_masked = x * mask
```

```
        x_hat = self(x_masked)
```

```
#####
#####
    loss = self.loss(x, x_hat)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss
```

ChatGPT

```
```python
class MaskedAutoencoder(Autoencoder):
 def __init__(self, input_size: int, hidden_sizes: List[int],
 activation_cls: nn.Module = nn.ReLU, mask_prob: float = 0.25):
 super().__init__(input_size, hidden_sizes, activation_cls)
 self.mask_prob = mask_prob

 def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:

#####
#####
 # Mask entries i.i.d. with probability `mask_prob` (0 -> masked out).
 # Reconstruct from the masked input, but compute loss ONLY on the
 # UNMASKED positions, as specified.

#####
#####
 self.train()
 mask = (torch.rand_like(x) > self.mask_prob).to(x.dtype) # 1=unmasked,
0=masked
 x_masked = x * mask
 x_hat = self(x_masked)

 # Masked MSE: sum over features on unmasked entries, mean over batch
 loss = F.mse_loss(x_hat * mask, x * mask, reduction="sum") / x.size(0)
```



```
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
return loss
```

```
...
```

**user**

## ## Training Autoencoders

In this section, you will learn how to train and evaluate autoencoders. After each training epoch, you will calculate the linear probe accuracy on the test split of your dataset.

To achieve this, you will first use your trained autoencoder to encode each example in the dataset  $x_i$  into its latent representation  $z_i$ . Then, you will use these latent representations  $z_i$ , along with their corresponding labels  $y_i$ , to train a simple linear classifier called a linear probe.

The linear probe accuracy is the classification accuracy of this linear classifier on the test split of the dataset. By calculating this accuracy, you can evaluate how well your autoencoder is able to capture the important features of the data and how useful those features are for downstream tasks like classification.

class Experiment:

```
def __init__(self, dataset, model: nn.Module,
 batch_size: int, num_classes: int, lr: float,
 probe_train_batch = "full", probe_train_epochs: int = 50):
 self.train_batch_size = batch_size
 self.eval_batch_size = 200
 self.dataset = dataset
 self.model = model.cuda()
 self.optimizer = optim.Adam(self.model.parameters(), lr=lr)
 self.num_classes = num_classes
 self.probe_train_batch = probe_train_batch
 self.probe_train_epochs = probe_train_epochs
```

```

def train(self, num_epochs: int) -> dict:
 self.model.train()
 train_losses, valid_losses, probe_accs = [], [], []

 pbar = tqdm(range(num_epochs))
 num_batches = self.dataset.num_train_samples // self.train_batch_size

 with torch.no_grad():
 valid_loss = self.get_loss(split="valid")
 valid_losses.append(valid_loss)
 probe_accs.append(
 self.evaluate_w_linear_probe(self.model.hidden_sizes[-1]))
 for epoch in pbar:
 for batch_idx in range(num_batches):
 x, y = self.dataset.get_batch(batch_idx, split="train")
 x, y = x.cuda(), y.cuda()
 loss = self.model.train_step(x, self.optimizer)
 train_losses.append(loss.item())
 pbar.set_description(f"Epoch {epoch}, Loss {loss.item():.4f}")
 with torch.no_grad():
 valid_loss = self.get_loss(split="valid")
 valid_losses.append(valid_loss)
 probe_accs.append(
 self.evaluate_w_linear_probe(self.model.hidden_sizes[-1]))

 return {
 "train_losses": train_losses,
 "valid_losses": valid_losses,
 "valid_accs": probe_accs
 }

```

```

def get_loss(self, split="train") -> float:
 """

```

Compute the average loss of the model on a specified dataset split.

Parameters:

- split (str, optional): The dataset split to compute the loss on.

Returns:

The average loss of the model on the specified dataset split.

"""

```
self.model.eval()
num_samples = self.dataset.get_num_samples(split=split)
num_batches = num_samples // self.eval_batch_size
assert num_samples % self.eval_batch_size == 0
losses = []
for batch_idx in range(num_batches):
 x, y = self.dataset.get_batch(batch_idx, split=split)
 x, y = x.cuda(), y.cuda()
 loss = self.model.get_loss(x)
 losses.append(loss.item())
return np.mean(losses)
```

def \_get\_model\_accuracy(self, classifier: nn.Module, split="test") -> float:

"""

Compute the accuracy of the model on a specified dataset split using a given linear classifier (a.k.a. a linear probe). This method is invoked by `eval\_w\_linear\_probe` that is defined below.

Parameters:

- classifier (nn.Module): The linear classifier to use for computing the accuracy.
- split (str, optional): The dataset split to compute the accuracy on.

Returns:

The accuracy of the model on the specified dataset split.

"""

```
self.model.eval()
num_samples, num_correct = 0, 0
num_batches = self.dataset.num_test_samples // self.eval_batch_size
assert num_samples % self.eval_batch_size == 0
for batch_idx in range(num_batches):
 x, y = self.dataset.get_batch(batch_idx, split="test")
```



```

####
```

```
TODO: Implement the following code in the evaluation loop to
calculate accuracy using the autoencoder and the given
classifier
```

```

####
```

```
x, y = x.cuda(), y.cuda()
z = self.model.encode(x)
y_hat = classifier(z)
preds = (y_hat.argmax(dim=1) == y).cpu().numpy()
num_samples += len(preds)
num_correct += np.sum(preds)
```

```

####
```

```
return num_correct / num_samples * 100
```

```
def evaluate_w_linear_probe(self, feats_dim) -> float:
```

```
 """
```

Evaluate the model using a linear probe on a small subset of the labeled data.

Parameters:

- feats\_dim (int): The number of features in the model's output.
- num\_epochs (int, optional): The number of epochs to train the linear probe for. Defaults to 10.

Returns:

The accuracy of the model computed using the linear probe.

```
 """
```

```
self.model.eval()
probe = nn.Linear(feats_dim, self.num_classes)
probe.cuda()
```

```

probe.train()
probe_opt = optim.Adam(probe.parameters(), lr=1e-3)

if self.probe_train_batch == "full":
 num_batches = (
 self.dataset.get_num_samples(split="valid")
 // self.eval_batch_size
)
else:
 num_batches = self.probe_train_batch

frozen_batch = []
with torch.no_grad():
 for batch_idx in np.random.permutation(num_batches):
 x, y = self.dataset.get_batch(batch_idx, split="valid")
 x, y = x.cuda(), y.cuda()
 feat = self.model.encode(x)
 frozen_batch.append((feat.cpu(), y.cpu()))

for epoch in range(self.probe_train_epochs):
 for feat, y in frozen_batch:
 feat, y = feat.cuda(), y.cuda()
 y_hat = probe(feat)
 loss = F.cross_entropy(y_hat, y)
 probe_opt.zero_grad()
 loss.backward()
 probe_opt.step()

Evaluate linear probe
probe.eval()
with torch.no_grad():
 accuracy = self._get_model_accuracy(classifier=probe)

return accuracy

```

**ChatGPT**



Here's a minimal fill for the TODO in `\_get\_model\_accuracy` that computes test accuracy using the autoencoder's encoder as a frozen feature extractor and the provided linear probe. It keeps things simple and uses `no\_grad()` for speed/memory:

```
```python
def _get_model_accuracy(self, classifier: nn.Module, split="test") -> float:
    """
    Compute the accuracy of the model on a specified dataset split using a
    given linear classifier (a.k.a. a linear probe).
    """
    self.model.eval()
    num_samples, num_correct = 0, 0
    num_batches = self.dataset.num_test_samples // self.eval_batch_size
    assert self.dataset.num_test_samples % self.eval_batch_size == 0 # optional

    for batch_idx in range(num_batches):
        x, y = self.dataset.get_batch(batch_idx, split="test")

        #####
        #####
        # TODO region: compute accuracy via encode -> classifier -> argmax

        #####
        #####
        with torch.no_grad():
            x, y = x.cuda(), y.cuda()
            z = self.model.encode(x) # (B, feat_dim)
            y_hat = classifier(z) # (B, num_classes)
            preds = y_hat.argmax(dim=1) # (B,)
            num_correct += (preds == y).sum().item()
            num_samples += y.size(0)

        #####
        #####
    return num_correct / num_samples * 100
...
```
```

> Note: that `assert` line previously referenced `num\_samples` before it was set; I changed it to check the dataset size instead to avoid a false assertion.

**user**

### ### Visualization: Training Curves

You have saved all the training logs for the autoencoder models with different feature dimensions. In this section, you will implement a function to visualize the training curves and accuracy using linear probes. This visualization will help you compare the performance of autoencoder models of different hidden sizes.

To begin with, you need to visualize the training curves of the **vanilla** autoencoder with latent representations of different feature dimensions. You will need to complete the following code to draw two plots:

- The x-axis of both plots should represent the training epochs, while the y-axis of the first plot should display the validation loss (reconstruction error) and the y-axis of the second plot should display the linear probe accuracy.
- Both plots should be line plots with four curves, each representing a feature dimension of 5, 20, 100, and 1000, respectively.
- Each curve should have a major line and an area around the line:
  - The major line should have one dot for each epoch showing the average validation loss/accuracy of that epoch over three runs.
  - The area should be filled between the minimum and maximum validation loss/accuracy of that epoch across the three runs.
  - The color of the line, dots, and area should be the same, with the area being translucent.
- Both plots should include axis labels (for the x and y axis), a legend, and a title.

Ensure that your implementation accurately reflects the requirements outlined above.

**Documents for reference:**

- [https://matplotlib.org/stable/api/\\_as\\_gen/matplotlib.pyplot.plot.html](https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html)
- [https://matplotlib.org/stable/api/\\_as\\_gen/matplotlib.pyplot.fill\\_between.html](https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.fill_between.html)

```
def plot_single(values, feats_dim):
```

```
#####
```

```
#####
```

```
 # TODO: Implement the following code to draw a single curve with a filled
 # area around it.
```

```
 #
```

```
 # Hint 1: `values` is a list of three lists, where each list corresponds to
```

```
 # one run and each entry in the list corresponds to an epoch
```

```
 # Hint 2: `feats_dim` is useful for showing legends
```

```
#####
```

```
#####
```

```
 x = range(0, hparams.num_epochs + 1)
```

```
 y_avg = np.mean(values, axis=0)
```

```
 y_min = np.amin(values, axis=0)
```

```
 y_max = np.amax(values, axis=0)
```

```
 plt.plot(x, y_avg, marker='o', markersize=4, label=f"d = {feats_dim:3d}")
```

```
 plt.fill_between(x, y_min, y_max, alpha=0.4)
```

```
#####
```

```
#####
```

```
Visualize valid losses
```

```
for model_idx in valid_losses.keys():
```

```
 if model_idx[0] == 'vanilla':
```

```
 plot_single(valid_losses[model_idx], model_idx[1])
```

```
#####
```

```
#####
```

```
TODO: Implement the following code to draw legends, axis labels, and the title
```

```
#####
```

```
#####
```

```
plt.legend()
```

```
plt.xlabel("epoch")
```

```
plt.ylabel("valid loss")
```

```

plt.title("Reconstruction Error")
#####
#####
plt.show()

Visualize valid (linear probe) accuracy
for model_idx in accuracy.keys():
 if model_idx[0] == 'vanilla':
 plot_single(accuracy[model_idx], model_idx[1])
#####
#####
TODO: Implement the following code to draw legends, axis labels, and the title
#####
#####
plt.legend()
plt.xlabel("epoch")
plt.ylabel("accuracy")
plt.title("Linear Prob Accuracy")
#####
#####
plt.show()

```

```
TO_SAVE["vis_fn"] = inspect.getsource(plot_single)
```

## ChatGPT

Here's a clean implementation that matches all the plotting requirements (same color for line/markers and the translucent band; dots per epoch; min-max band across the three runs).

```

```python
def plot_single(values, feats_dim):

#####
#####
    # Draw a single curve with dots (epoch-wise mean) and a translucent band

```



```
# (epoch-wise min-max across the 3 runs).
# `values`: list of 3 lists/arrays shaped [num_epochs + 1]

#####
#####
vals = np.asarray(values, dtype=float)          # (3, T)
x = np.arange(vals.shape[1])                    # epochs 0..T-1
y_avg = vals.mean(axis=0)
y_min = vals.min(axis=0)
y_max = vals.max(axis=0)

(line,) = plt.plot(x, y_avg, marker='o', markersize=4, linewidth=1.8,
                   label=f"d = {feats_dim:3d}")
color = line.get_color()
plt.fill_between(x, y_min, y_max, color=color, alpha=0.2)
...

```python
Visualize valid losses
for model_idx in valid_losses.keys():
 if model_idx[0] == 'vanilla':
 plot_single(valid_losses[model_idx], model_idx[1])
#####
#####
Legends, axis labels, and title
#####
#####
plt.legend(title="Latent dim")
plt.xlabel("Epoch")
plt.ylabel("Validation loss (reconstruction error)")
plt.title("Vanilla Autoencoder: Reconstruction Error vs Epoch")
plt.tight_layout()
#####
#####
plt.show()

Visualize valid (linear probe) accuracy
```

```

for model_idx in accuracy.keys():
 if model_idx[0] == 'vanilla':
 plot_single(accuracy[model_idx], model_idx[1])
#####
#####
Legends, axis labels, and title
#####
#####
plt.legend(title="Latent dim")
plt.xlabel("Epoch")
plt.ylabel("Linear probe accuracy (%)")
plt.title("Vanilla Autoencoder: Linear Probe Accuracy vs Epoch")
plt.tight_layout()
#####
#####
plt.show()

TO_SAVE["vis_fn"] = inspect.getsource(plot_single)
'''

```

This will produce two line plots (four curves each for  $d \in \{5, 20, 100, 1000\}$ ), with epoch-wise means marked by dots and min–max shaded bands using the same color as each curve.

**user**

#### Question

In your written assignment submission, please answer the following question:  
**\*\*How does changing the latent representation size of the autoencoder affect the model's performance in terms of reconstruction accuracy and linear probe accuracy? Why?\*\*** Hint: each datapoint in the synthetic dataset has 100 dimensions, with 20 high-variance dimensions that affect the class label.

**ChatGPT**



**\*\*Short answer (with reasoning tied to the hint):\*\***

Let  $d=100$  and suppose the data covariance has  $20$  high-variance directions that also carry the label signal. For a (vanilla) autoencoder trained with MSE and no additional regularization:

#### ### Reconstruction accuracy

- As the latent size  $k$  **increases**, reconstruction error **decreases monotonically** (more capacity  $\rightarrow$  less information discarded).
- In the linear case, the optimal AE spans the top  $k$  principal components (PCA). The reconstruction error equals the sum of the discarded eigenvalues:

$$\mathcal{L}_{\text{recon}}(k) = \sum_{j>k} \lambda_j,$$

where  $\lambda_1 \geq \lambda_2 \geq \dots$  are eigenvalues of the data covariance.

Thus:

- $k=5 < 20$ : many high-variance directions are dropped  $\rightarrow$  **highest** error.
- $k=20$ : all 20 high-variance directions kept  $\rightarrow$  **large drop** in error.
- $k=100$  (undercomplete but close to  $d$ ) and  $k=1000$  (overcomplete): can nearly copy the input  $\rightarrow$  **very low** error (overcomplete may even memorize).

#### ### Linear probe accuracy

- The label depends on those **20** high-variance directions. A linear probe trained on latent codes  $z$  will do well **iff** the AE preserves these directions.
  - $k=5 < 20$ : the AE must compress multiple informative directions into fewer coordinates  $\rightarrow$  **label information is lost**  $\rightarrow$  **low** probe accuracy.
  - $k \approx 20$ : the code can carry **all** label-relevant directions  $\rightarrow$  **sharp jump** to **high** accuracy; often near its plateau.
  - $k \gg 20$  (e.g.,  $k=100$  or overcomplete  $k=1000$ ): the AE also preserves **low-variance/nuisance** directions. A linear probe **can** ignore them in principle, but in practice:

- Higher dimensional  $z$  increases sample complexity and risk of overfitting without strong regularization.
  - The unsupervised objective may **entangle** signal with noise, so the linear boundary on  $z$  can be **harder to learn** from finite data.
- Result: probe accuracy typically **plateaus** around  $k \approx 20$  and may **stay similar or dip slightly** as  $k$  grows far beyond 20.

### Takeaways

- **Reconstruction** reliably improves with larger  $k$  (PCA intuition: keep more variance  $\rightarrow$  lower MSE).
- **Linear-probe accuracy** improves rapidly until  $k$  reaches the **intrinsic discriminative dimension** ( $\sim 20$  here), then **saturates** and can **degrade** slightly for very large/overcomplete codes unless you add regularization (e.g., denoising/masking, sparsity, weight decay) to bias the representation toward the label-carrying subspace.

**user**

### Nonlinear Dimensionality Reduction on MNIST

In the previous section, we observed that there is no advantage in terms of linear probe accuracy when we perform dimension reduction. The reason for this is that we use the entire *labeled* validation dataset to train the linear probe, rendering the use of autoencoders and self-supervised learning less useful in cases where we have ample labeled training data.

In this part, we will consider a different scenario where we have an abundance of *unlabeled* training data, but only a limited number of *labeled* examples. Specifically, we will train a non-linear autoencoder on the MNIST dataset using all images in the dataset, but only **200** labeled examples will be used to train the linear probe.

Your task is to train a non-linear autoencoder on the MNIST dataset. The objective is to achieve a few-shot linear probe accuracy of at least **79%** on the last epoch, averaged over two random runs. You may use any type of autoencoder that you have previously implemented, choose any latent

representation sizes, and your grade will be evaluated on a linear scale, ranging from 0 to the maximum score.

The validation accuracy achieved on the last epoch should range from 70% to 79%. If the accuracy is less than 70%, you will receive a score of 0, and if it is greater than 79%, you will receive the full score for this autograding item.

```
Do not change these
NUM_REPEATS = 2
input_dims = 28 * 28
num_classes = 10
data_splits = [0.9, 0.1]
```

```
#####
#####
TODO: Set the hyperparameters
#####
#####
hparams = argparse.Namespace(
 batch_size=100,
 num_epochs=25,
 hidden_dims=[100, 100],
 activation="ReLU",
 lr=5e-4,
 noise_std=0.5,
 mask_prob=0.25
) # 79.95
#####
#####

#####
#####
TODO: Define a function to build the model. You are encouraged to experiment
```

```

with different types of autoencoders
#####
#####
def build_model():
 return MaskedAutoencoder(
 input_dims,
 hparams.hidden_dims,
 activation_cls=getattr(nn, hparams.activation)
)
#####
#####

dataset = MNIST(hparams.batch_size, data_splits)

feats_dim = hparams.hidden_dims[-1]
train_loss, valid_loss, acc = [], [], []
for expid in range(NUM_REPEATS):
 _set_seed(expid * 227)
 model = build_model()
 experiment = Experiment(
 dataset,
 model,
 batch_size=hparams.batch_size,
 num_classes=num_classes,
 lr=hparams.lr,
 probe_train_batch=1, # 1 batch = 200 examples
 probe_train_epochs=1000
)
 _set_seed(1998 + expid * 227)
 train_stats = experiment.train(num_epochs=hparams.num_epochs)
 train_loss.append(train_stats["train_losses"])
 valid_loss.append(train_stats["valid_losses"])
 acc.append(train_stats["valid_accs"])

TO_SAVE["train2"] = {
 "train_loss": train_loss,
 "valid_loss": valid_loss,

```



```
"acc": acc,
"hparams": hparams.__dict__,
"build_model_fn": inspect.getsource(build_model),
}
```

```
print("Avg Accuracy:", np.array(acc).mean(axis=0)[-1])
```

## ChatGPT

```
```python
#####
#####
# TODO: Set the hyperparameters
#####
#####
hparams = argparse.Namespace(
    batch_size=100,
    num_epochs=25,
    hidden_dims=[100, 100], # nonlinear (2-layer) encoder; latent dim = 100
    activation="ReLU",
    lr=5e-4,                # Adam lr
    noise_std=0.5,          # (unused for masked AE; kept for completeness)
    mask_prob=0.25          # 25% feature masking during training
) # empirically reaches ~79–80% few-shot probe acc on MNIST (last epoch, avg
of 2 runs)
#####
#####

#####
#####
# TODO: Define a function to build the model. You are encouraged to experiment
# with different types of autoencoders
#####
#####
def build_model():
    return MaskedAutoencoder(
```

```

        input_dims,
        hparams.hidden_dims,
        activation_cls=getattr(nn, hparams.activation)
    )
#####
#####

'''

```

user

Please, read https://en.wikipedia.org/wiki/K-means_clustering about the Kmeans algorithm.

In this problem, we will show how interpreting a dataset as a graph may result in obtaining an elegant clustering solution. We have an input dataset that we wish to cluster in 3 apparent classes.

We provide the synthetic dataset of 2000 points described below where the T_matrix is just a 2D transformation matrix:

Let's now interpret every single point in the provided dataset as a node in a graph. Our goal is to find a way to relate every node in the graph in such a way that they points that are closer together maintain that relationship while points that are far are explicitly identified.

lots of points are close to each other and kmeans is missing it. representing as the graph unveils the relation.

One way to capture such relationship between points (nodes) in a graph is through the Adjacency matrix. Typically, a simple adjacency matrix between nodes of an undirected graph is given by:

```


$$A_{i,j} = \begin{cases} 1 & \text{if there is an edge between node } i \text{ and node } j, \\ 0 & \text{otherwise.} \end{cases}$$


```

```

\end{aligned}
\right.
\end{equation}
$$

```

In this problem, we will use the weighted distances between points instead as a similarity measure. Write a function that takes in the input dataset and some coefficient gamma which returns the adjacency matrix A.

$$A_{i,j} = e^{\gamma \|x_i - x_j\|^2}$$

where x_i and x_j represent each point in the provided dataset. You may find the `distance` module from `scipy.spatial` useful.

```

def get_adjacency_matrix(gamma, X):
    # TODO: fill in your code here
    # adjacency_matrix = ?

    return adjacency_matrix

```

ChatGPT

```

```python
from scipy.spatial import distance
import numpy as np

def get_adjacency_matrix(gamma, X):
 """
 Construct weighted adjacency: A_ij = exp(gamma * ||x_i - x_j||^2).
 By convention for graph adjacency we zero the diagonal (no self-loops).
 """
 D2 = distance.cdist(X, X, metric='sqeuclidean') # pairwise ||xi - xj||^2
 adjacency_matrix = np.exp(gamma * D2)
 np.fill_diagonal(adjacency_matrix, 0.0)
 return adjacency_matrix
...

```

**user**

The degree matrix of an undirected graph is a diagonal matrix which contains information about the degree of each vertex. In other word, it contains the number of edges attached to each vertex and it is given by:

```
$$D_{i, j}=\begin{equation}
\left\{
\begin{aligned}
&\deg(v_{\{i\}}): \text{if } i == j, \\
&0: \text{otherwise.}
\end{aligned}
\right.
\end{equation}
$$
```

where the degree  $\deg(v_{\{i\}})$  of a vertex counts the number of times an edge terminates at that vertex. Note that in the traditional definition of the adjacency matrix, this boils down to the diagonal matrix in which element along the diagonals are column-wise sum of the adjacency matrix. Using the same idea, write a function that takes in the adjacency matrix as argument and returns the inverse square root of degree matrix.

```
def get_degree_matrix(adjacency_matrix):
 # TODO: fill in your code here
 # degree_matrix = ?

 return degree_matrix
```

**ChatGPT**

```
```python
import numpy as np

def get_degree_matrix(adjacency_matrix):
    """
```


Return $D^{-1/2}$, where D is the degree matrix of an undirected (weighted) graph.

By definition here, degrees are the column-wise sums of the adjacency matrix.

For isolated nodes (degree == 0), set the corresponding $D^{-1/2}_{ii} = 0$ to avoid inf.

```
"""
A = np.asarray(adjacency_matrix)
deg = A.sum(axis=0) # column-wise sums -> degrees (equals row-sum if A is
symmetric)
with np.errstate(divide='ignore'):
    inv_sqrt_deg = np.where(deg > 0, 1.0 / np.sqrt(deg), 0.0)
    degree_matrix = np.diag(inv_sqrt_deg.astype(A.dtype, copy=False))
    return degree_matrix
...

```

user

Using $\gamma = 7.5$, compute the symmetrically normalized adjacency matrix A , degree matrix D and the matrix $M = D^{-1/2} A D^{-1/2}$

adjacency_matrix = ?

degree_matrix = ?

M = ?

ChatGPT

```
```python
```

```
Using $\gamma = 7.5$
```

```
adjacency_matrix = get_adjacency_matrix(7.5, X)
```

```
Note: per our earlier function, this returns $D^{-1/2}$ (inverse sqrt degree), not D itself
```

```
degree_matrix = get_degree_matrix(adjacency_matrix)
```

```
Symmetric normalization: $M = D^{-1/2} A D^{-1/2}$
```



```
M = degree_matrix @ adjacency_matrix @ degree_matrix
```

```
...
```

**user**

Using SVD decomposition, write a function that select the first 3 vectors in the matrix U and perform the same KMeans clustering used above to cluster them. What do you observe? Did it work? If so explain why, if not, explain not not.

**ChatGPT**

Here's a compact spectral-clustering helper that (i) computes the SVD of your symmetric normalized affinity  $(M)$ , (ii) takes the first 3 left singular vectors  $(U_{[:, :3]})$  as the spectral embedding, (iii) row-normalizes them (standard in spectral clustering), and (iv) runs the same K-Means to get 3 clusters.

```
```python
import numpy as np
from sklearn.cluster import KMeans

def spectral_kmeans(M, n_clusters=3, n_components=3, normalize_rows=True,
                    random_state=0):
    """
    Spectral clustering via SVD on symmetric normalized affinity  $M = D^{-1/2} A D^{-1/2}$ .
    Returns KMeans labels on the top-n_components left singular vectors.
    """
    # SVD:  $M = U S V^T$  ; for symmetric M,  $U == V$  and columns of U are
    # eigenvectors
    U, S, Vt = np.linalg.svd(M, full_matrices=False)
    U_k = U[:, :n_components]          # (N, 3)

    if normalize_rows:
        norms = np.linalg.norm(U_k, axis=1, keepdims=True) + 1e-12
        U_k = U_k / norms

    km = KMeans(n_clusters=n_clusters, n_init=10, random_state=random_state)
```

```

    labels = km.fit_predict(U_k)
    return labels, U_k, S[:n_components]
...

```

Usage (given your previously computed `M`):

```

```python
labels, U3, singvals = spectral_kmeans(M, n_clusters=3, n_components=3,
normalize_rows=True, random_state=42)
...

```

---

### What do you observe? Did it work—why or why not?

- **If it works (typical when similarity is well-formed):**

You should see clean separation into the 3 apparent clusters, even when vanilla K-Means on the raw  $\mathbb{R}^2$  points fails. Reason: the graph affinity  $A$  (and its symmetric normalization  $M = D^{-1/2} A D^{-1/2}$ ) captures **manifold/neighbor** structure. The top eigenvectors of  $M$  are approximately **piecewise constant on clusters** (think: indicators), so points within the same cluster map to similar rows in  $(U_{[:, :3]})$ . K-Means then operates in this “unfolded” spectral space, where clusters are (nearly) linearly separable.

- **If it doesn't work (common pitfalls):**

1) **Wrong similarity sign/scale.** A Gaussian/RBF affinity should **decrease** with distance:  $A_{ij} = \exp(-\gamma \|x_i - x_j\|^2)$ . If you used  $(+\gamma)$  (grows with distance), far points look *more similar*, which breaks the graph and can cause numeric blow-ups. Fix by using a **negative sign** (or pass a negative  $\gamma$ ).

2)  **$\gamma$  too small/large.** Too small  $\rightarrow$  nearly constant  $A$  (no structure). Too large  $\rightarrow A$  nearly zero except for tiny neighborhoods (graph fragments). Tune  $\gamma$  or use a k-NN graph.

3) **No row normalization before K-Means.** Without it, scale differences across rows of  $U$  can hurt clustering; row-normalization (as above) is standard.

4) **Dense fully-connected noise.** If the underlying clusters aren't well captured by local affinities (or noise is heavy), consider k-NN adjacency and/or sparsifying  $A$ .

When the affinity is constructed properly (e.g., RBF with a sensible  $\gamma$  or k-NN graph) and  $M$  is used, **spectral clustering + K-Means on  $U$**  typically recovers the intended 3 clusters, explaining why the graph view can succeed where plain K-Means in the input space fails.

## user

Now let's think of the Adjacency obtained above as the transition Matrix in of a Markov Chain. To do so,  $A$  needs to be a proper stochastic matrix which means that the sum of the element in each column must add up to 1. Write a function that takes in the matrix  $M$  and returns  $M_{\text{stochastic}}$ , the stochastic version of  $M$ ; compute the stochastic matrix

```
def stochastic_matrix_converter(M):
 # TODO: fill in your code here
 # degree_matrix = ?

 return M_stoch
```

## ChatGPT

```
```python
import numpy as np

def stochastic_matrix_converter(M):
    """
    Convert a matrix M to a column-stochastic matrix M_stoch
    (each column sums to 1). Columns that sum to 0 are left as all-zeros.
    """
    M = np.asarray(M, dtype=float)
    col_sums = M.sum(axis=0, keepdims=True)          # shape (1, N)
    safe_den = np.where(col_sums != 0.0, col_sums, 1.0) # avoid divide-by-zero
    M_stoch = M / safe_den
```

```
    return M_stoch
...
```

Usage:

```
```python
```

```
M_stoch = stochastic_matrix_converter(M)
```

```
Optional check (allowing tiny numerical tolerance):
```

```
assert np.allclose(M_stoch.sum(axis=0), 1.0, atol=1e-8)
```

```
...
```